

1. Two stacks in a single Array

Two stacks can be implemented efficiently in a single array by allowing one stack to grow from the left end and the other from the right end

- Stack 1:- starts from index 0 and grows towards the right
- Stack 2:- starts from index $n-1$ and grows towards the left
- Overflow condition:- Occurs only when $top_1 + 1 == top_2$

Comparison of fixed Partitioning and Dynamic Allocation

Feature	Fixed Partitioning	Dynamic Allocation
Memory allocation	Array divided into two fixed halves	Both stacks share the array
Space efficiency	lower (unused space may be wasted)	Higher (unused space can be used by either stack)
overflow	one stack may overflow even if the other has free space	overflow only when the entire array is full
Time complexity	Push/Pop: $O(1)$	Push/Pop: $O(1)$
Flexibility	less flexible	more flexible

Analysis:-

• Fixed Partitioning:-

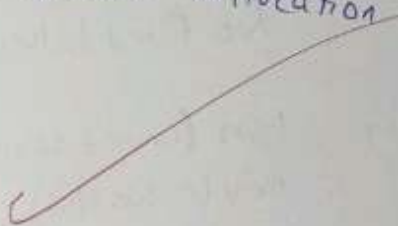
simple to implement but wastes memory if one stack is small and the other becomes full.

• Dynamic allocation:-

Maximizes memory utilization and delays overflow until the array is completely full.

• Operational complexity:-

Both approaches perform push and pop operations in $O(1)$ time but dynamic allocation provides better space efficiency.



2. Queue-Based Ticketing System:-

A simple Queue follows the FIFO (First In, First out) principle, where customers are served in the order they arrive.

A Priority Queue allows VIP customers to be served before normal customers based on priority.

Comparison of simple Queue and priority Queue:-

Feature	simple Queue	priority Queue
service order	FIFO	Based on priority
waiting time	Equal for all	lower (normal customers increased for longer)
Enqueue complexity	$O(1)$	Reduced for $O(\log n)$ Heap (or) $O(n)$ (Array)
Dequeue complexity	$O(1)$	$O(\log n)$ heap
Fairness	High	lower (normal customers may wait longer)

Analysis:-

• Simple Queue:-

Fair and easy to implement but cannot prioritize urgent customers.

• Priority Queue:-

Improves service for VIP customers but may increase waiting time for regular customers (starvation can occur if VIP's keep arriving).

• Complexity:-

Priority queues require extra processing to maintain priorities, making them computationally more expensive than simple queues.

Conclusion:-

- Use a simple queue when fairness is the primary goal.
- Use a priority queue when urgent (or) VIP customers must receive faster service, even at the cost of increased complexity and reduced fairness.